

RBT AI 训练营 · 第 1 周课件

立项 + 全套 AI CLI 工具箱 + 跑通第一个 AI-driven commit

周四 15:00-17:00 · 概念讲解 + Live Demo + Q&A

周五 15:00-17:00 · Lab: 每人立项、装环境、跑通第一个 commit

课前必读（学员请在周四上课前看完）

这门课是什么

这是一个 **10 周的 AI Multi-Agent & Fullstack 培训**。它的目标**不是**教你一门编程语言、也不是教你具体某个框架。这些你已经在 UNSW 的 COMP9021 / COMP9311 / COMP6080 里学过了。

它的目标是教你：

1. 怎么用 **AI 协作** 完成一个真实的全栈项目
2. 怎么用 **multi-agent 工作流** 让一个人达到一个小团队的产出
3. 怎么走完一遍**工程流程**：PRD → 数据库 → 后端 → 前端 → 部署 → review → 迭代

这门课不是什么

- 不是"prompt 课"：我们不背 prompt 模板
- 不是"工具课"：我们不为了学工具而学工具
- 不是"理论课"：每一个工具/概念都必须当周落地到你自己的项目里

你为什么在这里

因为软件工程师的工作方式正在改变。会写代码已经不是稀缺技能——会和 **AI 协作** 把一个想法变成可运行、可部署、可维护的系统才是。

10 周后你应该带走两样东西：

1. 一个**真实部署**、可演示、能讲故事的全栈项目
 2. 一套从此可以迁移到任何项目的 **AI-native 开发肌肉记忆**
-

第一部分：课程总览

1.1 培训背景

我们做这个训练营的原因有三个：

1. **AI 工具进化的速度远超传统教学** —— 学校来不及讲 Claude Code / Codex CLI / agentic workflow 这些工具，但行业已经在用。这门课补上这个 gap。
2. **会写代码 ≠ 会做项目** —— UNSW 的课程教你写 function、写 SQL、写 React 组件。但真正部署一个完整项目涉及 PRD、架构、部署、CI、review、迭代——这些是空白的。
3. **AI 时代需要"产品视角"** —— 单纯能写代码的工程师价值在降低；能用 AI 高效产出 + 懂产品 + 懂用户 的工程师价值在飙升。后半段课程会让你接触到 AI 编程产品和创业生态。

1.2 课程主线项目

每人一个完整的全栈项目，要求：

- 必须有数据库（PostgreSQL / SQLite / D1 等任选）
- 必须有后端（Python / Node.js / Go / Rust 等任选）
- 必须有前端（React / Vue / Svelte / 静态页面等任选）
- 必须可部署上线（Cloudflare Pages/Workers 推荐，其他平台也可）

选什么项目：建议挑你自己真的想用的东西。学习类、工具类、社区类、个人爱好类都行。不建议：

- 纯算法 demo（没有前后端）
- 已经有 50 个开源克隆品的"todo list"
- 超出 10 周能完成的"做下一个 ChatGPT"

W1 周五立项时讲师会和你 1:1 校准方向。

1.3 10 周节奏一览

周	主题	你这周产出什么
W1	立项 + 工具箱 + 第一个 AI commit	repo、CLAUDE.md、AGENTS.md、README v0、第一个 commit
W2	PRD + GitHub Issues + 造一个 skill	PRD.md、≥8 个 issue、第一个自定义 skill
W3	数据库 schema + 后端骨架 (TDD)	migration、≥2 个 endpoint、CI 通过
W4	前端骨架 + 前后端联调	可点击页面、API contract、前端测试
W5	第一次部署上线 + AI review 接入	公网 URL、CI 自动部署、第一次 AI review report
W6	中期 retrospective + AI 编程工具产品景观	retro 笔记、AI 工具景观对比笔记
W7	多 AI 互审实战 + AI 公司模型策略讨论	Claude × Codex 互审报告、模型策略笔记
W8	复杂功能 + AI 产品 design review	中等复杂度功能、产品 review 报告
W9	完整 review pipeline + 工程师角色讨论	清零 Critical、同学互评报告
W10	Polish + Demo Day + AI 创业讨论	v1.0 部署、Demo Day 演示

节奏特点：前 5 周打地基（项目从 0 到上线），后 5 周深化 + 大量讨论（接触 AI 产品生态和行业视角）。

1.4 周内固定结构

每周固定节奏：

1	周四 15:00-17:00	2h	讲新工具/新概念 + 现场 demo + Q&A
2	周五 15:00-17:00	2h	Lab: 把当周工具应用到自己项目 + 1:1 答疑
3	周末-周三	~5h	自学 + 推进项目 + 写 report.md
4	周三 23:59	—	截止提交 PR (含 report.md, 见 §1.7)
5	周四开课前	—	AI review agent 自动跑完所有 PR, 结果在课上讲评

每周大约 4 小时正课 + 5 小时自学 = 9 小时。10 周共约 90 小时。

1.5 学习成果（10 周后你会什么）

类别	具体能力
工具熟练度	Claude Code / Codex CLI 命令行参数、CLAUDE.md / AGENTS.md 写作、自定义 skill 创建、settings.json 配置
multi-agent 协作	区分 backend / frontend / test agent 职责；让多个 AI 互审；让 AI 跑完 issue → PR → review 流程
工程流程	PRD 设计、spec-driven、TDD、git-issues 工作流、GitHub Actions、Cloudflare 部署、AI review
产品视角	能拆解 AI 产品设计、能对比 AI 公司模型策略、能为自己项目设计差异化定位
可迁移肌肉记忆	任何新项目第一天就知道怎么搭 CLAUDE.md / 怎么拆 issue / 怎么部署

1.6 评估机制

每周提交 PR → AI review agent 自动跑 → 输出结构化报告（Critical / Major / Minor / Suggestion）+ 4 项 rubric 评分：

- 里程碑达成 40%
- 代码质量 25%
- 流程合规 20%
- AI 协作质量 15%

W1-W3 仅反馈、不计分（让你适应节奏）。W4-W9 每周计分，进入期末总分 70%。Demo Day 占 30%。

AI review 误判可以申诉，讲师每周抽查 ≥ 20% review 结果做校准。

1.7 每周必交：report.md（你的工作 + 学习日志）

这是这门课最重要的硬性要求之一，从 W1 起就要交。

每周三 23:59 提交 PR 时，必须随 PR 附上一份 `report.md`（建议路径：`docs/reports/W<N>-<your-id>.md`，每周新建一份，不覆盖旧的）。

为什么必须写

在 AI 时代，"备忘录"的价值不是给老师交差，而是：

1. **备忘录是你给 AI 的上下文** —— 你这周做了什么、踩了什么坑、为什么这么选，下一次和 AI 协作时把 report 喂回去，AI 立刻"记得"项目历史。**写得越好，下一周 AI 帮你越准。**
2. **写下来才是真正学到** —— 看会了不等于学会了。逼自己用文字复述本周的工具/概念/决策，能暴露 50% 你以为懂其实没懂的部分。
3. **个人成长档案** —— 10 周后回看 W1 的 report，你会看见自己从"怎么连 git 都不会用 Claude"到"半小时跑通一个 feature"的真实轨迹。这是你简历最好的素材。
4. **项目的 changelog / decisions log** —— report 累积起来就是项目的口述史。未来你或别人想知道"为什么 W4 改了数据库 schema"，翻 report 比翻 commit history 快。
5. **AI 协作质量的证据** —— 评分 rubric 中 "AI 协作质量 15%" 主要靠 report 来评估。不写 report = 自动丢分。

`report.md` 必须包含的板块

```
1  # Week N Report - <your name / id>
2
3  ## 1. 本周做了什么（事实清单）
4  - 列出本周完成的具体事情：装了什么工具、写了哪些代码、修了哪些 bug
5  - 越具体越好。不要写"学习了 Claude Code"，要写"装了 Claude Code，跑通了第一个
    prompt，遇到 x 错误，靠 y 解决"
6
7  ## 2. 本周学到了什么（知识 + 反思）
8  - 我学到的新概念 / 新工具 / 新工作方式
9  - 我之前以为对、但这周发现是错的
10 - 我最 aha 的瞬间
11 - 我还没完全搞懂的（留给下周）
12
13 ## 3. AI 协作记录
14 - 我这周用 AI 最爽的一次是什么（描述 prompt + AI 的输出 + 我怎么用）
15 - AI 这周最让我失望/坑我的一次是什么（描述 + 我怎么发现的 + 怎么修的）
16 - 我现在和 AI 协作的"个人套路"：_____
17
18 ## 4. 决策记录（可选但鼓励）
19 - 这周我做了哪些技术/产品上的选择？为什么？
```

```
20 - 例：选了 SQLite 不是 Postgres, 因为 ____
21
22 ## 5. 下周打算
23 - 1-3 件具体的事
24
25 ## 6. 资源沉淀
26 - 这周看的链接 / 视频 / 文章 (标题 + URL + 一句话评价)
27
28 ## 7. 求助 / 卡点 (如果有)
29 - 我现在最想问讲师的一个问题
30 - 我卡了 ≥ 30 分钟还没解决的事
```

写作要求

- **真实** —— 不要写客套话和模板腔。AI 一眼能看出来。讲师也能。
- **具体** —— "学习了 Git" ❌, "学到 `git reset --soft HEAD~1` 可以撤销 commit 但保留改动" ✅
- **简短** —— 300-800 字。不要凑字数。
- **诚实** —— 卡点、失败、不懂的，全部写。这是评分时最被尊重的部分。
- **可读** —— 一个陌生人看完应该能复述出你这周的故事。

一个反例（不要这样写）

本周我学习了 Claude Code 和 Codex CLI，了解了 CLAUDE.md 文件的作用，掌握了 git 的基础命令，对 AI 协作有了更深的理解。

—— 这种文字看完等于没看。改成：

本周装了 Claude Code，第一次跑 `claude` 卡在登录，原因是公司 VPN 拦了 anthropic.com。换网络后跑通。写 CLAUDE.md 时一开始把项目目标写得太抽象（"做一个好用的工具"），后来按讲师建议改成具体场景（"给 UNSW 学生找室友的小程序"），Claude 写出来的代码立刻贴合多了。

—— 这才是有信息密度的 report。

第二部分：AI-native 开发理念

2.1 三个 mindset shift

Shift #1: 你不是写代码，你是和 AI 协作产出代码

旧模式：打开编辑器 → 想 → 敲代码 → 调试。

新模式：打开 CLI → 描述意图 → AI 写代码 → 你审、改、跑 → commit。

这意味着你的核心技能从"打字快"变成"描述清楚 + 审得仔细"。

Shift #2: 上下文是新的代码

AI 写得好不好，70% 取决于你给它的上下文。这就是为什么有 `CLAUDE.md` / `AGENTS.md`、为什么要写 PRD、为什么要拆 issue。这些文件不是"流程负担"，它们是 AI 的输入。

Shift #3: 流程取代"灵感"

旧时代靠"灵感 + 调试"。新时代靠可复制的流程——spec → plan → test → impl → review → merge → deploy。流程越规范，AI 越能帮你。流程越混乱（vibe coding），AI 越容易把你带进坑。

2.2 什么是 multi-agent

一个 agent = 一个有特定角色和上下文的 AI 实例。比如：

- **backend agent**: 拿到 issue → 设计 schema、写 endpoint、写测试
- **frontend agent**: 拿到 API contract → 写组件、调用 API、写 UI 测试
- **test agent**: 专注边界条件、regression、e2e
- **reviewer agent**: 审 PR，找 bug、找设计问题、给改进建议

它们的"上下文"通过 `CLAUDE.md` / `AGENTS.md` / 自定义 prompt 文件来设定。同一个 Claude/Codex CLI 可以扮演不同角色——区别在于你给它什么上下文。

W3-W4 你会真正体会到 multi-agent 的威力。

第三部分：Claude Code 入门

3.1 安装

```
1 npm install -g @anthropic-ai/claude-code
2 claude --version
```

第一次启动：

它会引导你登录（Anthropic 账号）。

3.2 核心命令行参数

参数	作用	什么时候用
<code>claude</code>	启动新会话	默认
<code>claude --continue</code>	继续最近一次会话	上次没结束、想接着干
<code>claude --resume</code>	列出会话历史、选一个继续	想找一个几天前的会话
<code>claude --model claude-opus-4-7</code>	指定模型	需要更强推理
<code>claude --permission-mode plan</code>	进入只读规划模式	让 AI 先看代码不动手
<code>claude --permission-mode acceptEdits</code>	自动接受编辑	信任 AI、跑长任务
<code>claude -p "your prompt"</code>	单次 prompt 模式（非交互）	脚本里调用 / CI 用
<code>claude --append-system-prompt "..."</code>	追加 system prompt	临时给一段额外指令

常用组合：

```
1 # 启动会话并继续上次
2 claude --continue
3
4 # 给 AI 一个一次性任务，做完就退出
5 claude -p "把 README 里的 typo 都修一下"
6
7 # 进入计划模式：AI 只能读，不能写
8 claude --permission-mode plan
```

3.3 CLAUDE.md：你和 AI 的共享备忘录

`CLAUDE.md` 是放在项目根目录的 markdown 文件。Claude Code 每次启动都会读它，作为这个项目的"持久上下文"。

什么放进去：

- 项目目标和定位（1-2 段话）
- 技术栈和关键依赖
- 项目结构（哪些目录干什么）
- 命令清单（怎么跑、怎么测、怎么部署）
- 项目特有的规范（比如"所有 endpoint 必须有测试"、"commit message 用 Conventional Commits"）
- 当前在做的事（current focus、blockers）

什么不放：

- 通用编程知识（AI 已经知道）
- 长篇 design rationale（放在 `docs/` 里、需要时让 AI 自己去看）
- 秘密 / API key

一个 minimal 模板：

```
1  # Project: <项目名>
2
3  ## What this is
4  <一段话讲清楚这个项目是什么、给谁用、解决什么问题>
5
6  ## Tech stack
7  - Backend: FastAPI + PostgreSQL
8  - Frontend: React + Vite
9  - Deploy: Cloudflare Pages + Workers
10
11 ## Project structure
12 - `backend/` - FastAPI app
13 - `frontend/` - React app
14 - `docs/` - PRD, design notes
15 - `.github/workflows/` - CI
16
17 ## Commands
18 - `make dev` - run all locally
19 - `make test` - run all tests
```

```
20 - `make deploy` - deploy to Cloudflare
21
22 ## Conventions
23 - Use Conventional Commits (feat:, fix:, docs:, ...)
24 - All endpoints require tests
25 - All PRs need green CI before merge
26
27 ## Current focus
28 W1: project scaffolding + first AI-driven commit
```

W1 的目标是写出 `CLAUDE.md` v0（短、不完美都没关系），之后每周迭代。

3.4 日常工作流

做一个新功能：

```
1 $ claude
2 > 给我看看 backend/ 的结构
3 > 我想加一个 /api/items 的 endpoint, 做法跟 /api/users 一样
4 > 先写测试
5 > 测试看起来对, 现在写实现
6 > 跑一下
7 > commit
```

让 AI 调研代码：

```
1 $ claude --permission-mode plan
2 > 我想加一个用户角色系统, 请先看看现在的 auth 是怎么做的, 告诉我影响哪些文件、有什么风险
```

让 AI 单次跑完一个小任务：

```
1 $ claude -p "把 README 的英文 typo 修一下, 输出修改 diff"
```

第四部分：Codex CLI 入门

4.1 Codex 是什么、为什么也要装

Codex CLI 是 OpenAI 出的命令行 AI 编码工具，定位和 Claude Code 类似。我们装两个，是因为：

1. **不同模型擅长不同事** —— Claude 在长上下文、对话、细致 review 强；OpenAI 模型在某些 reasoning / 算法任务强
2. **互审更可靠** —— 让 Claude 写、让 Codex review（或反过来），能抓到单个 AI 的盲区
3. **冗余** —— 一个挂了/慢了/限速了还有另一个

4.2 安装

```
1 | npm install -g @openai/codex
2 | codex --version
```

4.3 命令行参数对比 Claude

Codex 的命令结构和 Claude 类似但不完全一样：

Claude	Codex 对应	备注
<code>claude</code>	<code>codex</code>	启动交互
<code>claude --continue</code>	<code>codex --continue</code>	继续会话
<code>claude -p "..."</code>	<code>codex exec "..."</code>	非交互单次
<code>--permission-mode plan</code>	<code>--ask-for-approval / --full-auto</code>	权限模式语义略不同

详细参数请运行 `codex --help` 看一遍。课程不背所有参数，用到时查。

4.4 AGENTS.md：Codex 的 CLAUDE.md

`AGENTS.md` 是 Codex 项目级上下文文件，作用和 `CLAUDE.md` 完全平行。

两个文件怎么处理？三种策略：

策略	适合	怎么做
共享	大多数项目	<code>AGENTS.md</code> 只写一行 <code>See CLAUDE.md</code> （参考本仓库 <code>rbt_founder_brainstorm/CLAUDE.md</code> ）
互补	两个 AI 各有专长	<code>CLAUDE.md</code> 放 Claude 特定的指令、 <code>AGENTS.md</code> 放 Codex 特定的，通用部分写在一个 <code>docs/shared-context.md</code> 里、两边都引用
独立	不同 AI 跑完全不同流程	各写各的（少见）

W1 lab 我们用最简单的"共享"策略——`AGENTS.md` 写 `See CLAUDE.md` 就够了。

4.5 什么时候用 Codex 而不是 Claude

经验法则（不是绝对）：

- 默认主力用 **Claude Code**（课程基于它设计）
- 想找盲区/**second opinion** 时切 **Codex**（W7 深入）
- 重算法/数学/调试 **reasoning** 任务：可以同时让两个跑、对比
- 成本敏感、长上下文：Claude 通常更好
- 速度敏感、短交互：两个差不多，视模型选择

第五部分：工具箱速览

下面 4 个工具是"装上就受益"的 utility，**W1** 我们只介绍 + 安装、不深入。后续周需要时你自然会用上。

5.1 RTK（Rust Token Killer）—— 5 min

是什么：CLI 代理，自动重写一些命令，节省 60-90% 的 token。

为什么用：每次让 AI 跑 `git status`、`ls`、`grep` 等命令，原始输出可能几千 token，rtk 会自动过滤、保留关键信息。

怎么装：

```
1 # macOS
2 brew install rtk
3 rtk --version
4
5 # 验证：跑一下分析过去节省的 token
6 rtk gain
```

怎么用：装好就行。Claude Code 的 hook 会自动调用 rtk。透明加速。

详细文档：参见 `~/.claude/RTK.md`

5.2 superpowers —— 8 min

是什么：一套 "skill 元体系"。让 Claude Code 拥有可复用、可组合的 skill（类似可调用的子模块）。

为什么用：很多操作（brainstorming、TDD、写 plan、review）有最佳实践，superpowers 把这些封装成 skill，你打 `/<skill>` 就能调用。

怎么装：在 Claude Code 里：

```
1 /plugin install superpowers
```

或参考 superpowers 仓库 README。

几个最常用的 skill：

- `/brainstorming` —— 把模糊想法变成结构化设计
- `/writing-plans` —— 把设计变成可执行的计划
- `/test-driven-development` —— TDD 流程引导
- `/systematic-debugging` —— 系统化调试

W2 起每次写 PRD / 拆 issue / 跑 TDD 时，你会自然用到这些。

5.3 gstack —— 8 min

是什么：一组面向"开发-QA-部署"全流程的 skill 集合，包括 headless 浏览器（用于自动 QA）、design review、ship 流程等。

为什么用：

- `/browse` —— headless 浏览器，可以让 AI 打开你刚部署的页面、截图、点击

- `/qa` —— 自动 QA 测试 + 修 bug
- `/design-review` —— 设计审计
- `/ship` —— 一键打包 PR
- `/review` —— 上线前 code review

怎么装：参见 gstack 仓库 README。

W4 之后你做前端时 `/browse` 会非常有用。

5.4 codegraph —— 5 min

是什么：代码导航工具。把项目的依赖、调用关系建成图，让 AI 看大型代码库时不"瞎子摸象"。

为什么用：当项目超过 30 个文件，AI 单靠 grep 已经看不清楚"如果改这个会影响什么"。codegraph 让 AI 一眼看到调用链。

怎么装：参见 codegraph 仓库 README。

W8 做大 feature 时你会用到。W1 装上备着。

5.5 工具速记表（你只需要记这个）

工具	什么时候想起它	一句话
Claude Code	默认	主力 AI CLI
Codex	想找 second opinion	OpenAI 的对应工具
RTK	装一次后忘记	自动节省 token
superpowers	想做 brainstorming/plan/TDD	skill 元体系
gstack	想做 QA / 部署 / 浏览器测试	全流程 skill 集合
codegraph	项目大了、AI 找不到东西	代码导航图

第六部分：周四 Live Demo（讲师演示）

讲师将在大屏现场完成下面这个完整 walking skeleton。请认真看，周五 lab 你要重复一遍。

Demo 脚本

```
1 # 1. 创建 repo
2 mkdir my-demo-project && cd my-demo-project
3 git init
4 gh repo create my-demo-project --private --source=. --remote=origin
5
6 # 2. 写 CLAUDE.md (用 Claude 帮你写第一版)
7 claude
8 > 我要做一个 [描述你的项目]。技术栈打算用 FastAPI + React + Cloudflare。
9 > 请帮我写一份 minimal CLAUDE.md。
10
11 # 3. 写 AGENTS.md (共享策略)
12 echo "See CLAUDE.md" > AGENTS.md
13
14 # 4. 让 Claude 写 hello-world
15 claude
16 > 创建一个最小的 FastAPI app, 一个 /api/hello endpoint, 返回 {"msg":
    "hello"}
17 > 加一个最小的 React 前端, 调用这个 endpoint 并显示
18 > 加一个 Makefile, make dev 能同时跑前后端
19
20 # 5. 跑起来
21 make dev
22
23 # 6. 用 gstack 的 /browse 打开看看
24 claude
25 > /browse open http://localhost:3000 并截图
26
27 # 7. 让 Codex review 一下
28 codex
29 > 看看这个项目, 给我一份 review, 指出至少 3 个改进点
30
31 # 8. commit
32 git add -A
33 git commit -m "feat: initial scaffold with hello world"
34 git push -u origin main
35
36 # 9. 看 RTK 帮你省了多少 token
37 rtk gain
```

重点观察：

- AI 是怎么"读懂"你的项目意图的 (CLAUDE.md 的作用)

- 你说了什么、AI 写出了什么
- Codex review 抓到了哪些 Claude 没注意的
- commit message 是怎么写的

第七部分：周五 Lab 任务

7.1 立项要求

每人立项一个全栈项目，符合以下规则：

维度	要求
必须有	数据库 + 后端 + 前端 + 可部署
范围	10 周内 1 人能做完 v1.0
不能选	纯算法 demo、明显商业敏感、纯前端工具
推荐	你自己真的想用的工具或学习类小产品

讲师/助教在 lab 上半段做 1:1 立项 review，给绿灯/红灯/调整意见。

7.2 步骤清单（按顺序做）

Step 1：装环境

```
1 # 检查所有工具
2 claude --version      # 应输出版本号
3 codex --version      # 应输出版本号
4 rtk --version         # 应输出版本号
5 git --version
6 gh --version          # GitHub CLI
7 node --version        # >=18
8
9 # 装 superpowers (在 Claude Code 里)
10 claude
11 /plugin install superpowers
12
13 # 安装 gstack 和 codegraph (参考各自仓库 README)
```

截图保留：所有 `--version` 输出（W1 验收要交）

Step 2: 建 repo + 写 CLAUDE.md

```
1 mkdir <your-project> && cd <your-project>
2 git init
3 gh repo create <your-project> --private --source=. --remote=origin
```

让 Claude 帮你写 CLAUDE.md:

```
1 claude
2 > 我要做的项目是 [项目描述、目标用户、3 个核心功能]
3 > 技术栈打算用 [...]
4 > 请帮我写一份 minimal CLAUDE.md
```

修改 + 提交。

Step 3: 写 AGENTS.md

最简单:

```
1 echo "See CLAUDE.md" > AGENTS.md
```

Step 4: 写 README.md v0

至少包含:

- 项目名 + 一句话定位
- 目标用户
- 3 个核心功能 (imagined for v1.0)
- 技术栈意向
- 安装/运行/测试/部署的占位命令

可以让 Claude 帮你起草, 再自己修。

Step 5: 让 Claude 跑通第一个 hello-world commit

参考周四 demo, 让 Claude 写最小可运行的前后端 + 一个 endpoint + 一个页面 + 一个 commit。

不要追求完美, 追求"跑得起来 + 已经 commit + 已经 push"。

Step 6: 自查 + push

```
1 # 自查 checklist
2 ls CLAUDE.md AGENTS.md README.md      # 三个文件都在
3 git log --oneline                      # 至少 1 个 commit
4 git status                            # working tree clean
5
6 # push
7 git push -u origin main
```

7.3 当周产出（PR / push 截止：周三 23:59）

#	产出	必须
1	私有 repo（已 push 到 GitHub）	✓
2	CLAUDE.md v0	✓
3	AGENTS.md v0	✓
4	README.md v0（含目标用户、3 个核心功能、技术栈）	✓
5	至少 1 个由 AI 协助完成的 commit	✓
6	工具安装截图（每个工具 --version 输出）	✓
7	hello-world 跑通（本地能起、能看到响应）	✓
8	docs/reports/W1-<your-id>.md（学习日志，见 §1.7）	✓
9	部署？	✗ W5 才要求

第八部分：W1 验收标准（AI Review Agent 会检查的项）

项	通过标准
repo 存在 + 可访问	把 repo URL 加到学员名单
CLAUDE.md 存在	大于 200 字、含项目目标
AGENTS.md 存在	任何内容都行（哪怕 See CLAUDE.md）
README.md 存在	含目标用户、≥3 个核心功能、技术栈
至少 1 个 commit	commit message 格式合理（不是 "update" 这种）
工具箱安装	6 个工具的 --version 截图齐全
hello-world 运行	review agent 会真的尝试 clone + 跑
report.md 存在且合格	含 §1.7 列出的 7 个板块；300-800 字；不是模板腔；具体到能复述故事

W1 不计分，只反馈。但下面这些会让 W2 起的评估更顺利：

- CLAUDE.md 写得有结构 → 后续 AI 帮你写代码更准
- commit message 规范 → 历史更可读
- README 写得清楚 → 你周二 lab 演示自己项目时不卡

第九部分：课后自学清单（周末-周三）

必读：

1. Claude Code 官方文档 "Quickstart"
2. Codex CLI 官方文档（首页 + 基本命令）
3. 一篇 CLAUDE.md best practices（Anthropic 官方或社区都行）
4. 一篇 AGENTS.md best practices

浏览即可：

5. RTK 仓库 README + 跑一次 rtk gain 看你已经省了多少 token
6. superpowers 仓库 README + 列出你想试的 3 个 skill
7. gstack 仓库 README + 找一个你想试的 skill（比如 /browse）
8. codegraph 仓库 README（知道是干啥的就行）

鼓励试一下：

- 9. 用 Claude 给你的项目写一份"用户场景"（user journey）—— 想象一个真实用户怎么用你的产品
- 10. 用 Codex review 你这周写的 hello-world，看它能挑出什么

周三 23:59 前必做：

- 11. 写 docs/reports/W1-<your-id>.md （学习日志，见 §1.7）—— 这是每周硬性产出，不写直接丢分

附录 A：工具命令速查表

1	# Claude	
2	claude	# 启动
3	claude --continue	# 继续上次
4	claude --resume	# 选会话继续
5	claude --permission-mode plan	# 只读计划模式
6	claude -p "prompt"	# 非交互单次
7		
8	# Codex	
9	codex	# 启动
10	codex --continue	# 继续
11	codex exec "prompt"	# 非交互单次
12		
13	# RTK	
14	rtk --version	
15	rtk gain	# 看节省统计
16		
17	# GitHub CLI	
18	gh repo create <name> --private --source=. --remote=origin	
19	gh pr create --title "... " --body "... "	
20	gh pr list	
21		
22	# git	
23	git status / log --oneline / diff	
24	git add -A && git commit -m "... "	
25	git push -u origin main	

附录 B：常见问题 FAQ

Q: 我没有 Anthropic / OpenAI 账号能拿到 API key 吗?

A: [由 founder 决定：是否提供共享账号 / 是否报销 / 是否走学生免费额度] —— 课前讲师统一说明。

Q: 我用 Windows 行吗?

A: 强烈建议用 WSL2 + Ubuntu。原生 Windows 下很多 CLI 行为不一致，会浪费你时间。

Q: 我能用 VS Code 的 Claude / Codex 插件代替 CLI 吗?

A: 课程主线是 CLI。你可以同时用插件，但每周产出必须能用 CLI 验证（因为评分时 review agent 走 CLI 路径）。

Q: 项目可以两个人合作吗?

A: 默认每人一个。如果想 2 人组队，W1 立项时和讲师商量，组队学员要求贡献量翻倍。

Q: 我项目方向被否了怎么办?

A: W1 周五 lab 期间会校准。讲师/助教会给具体调整建议。10 周内能完成是硬约束。

Q: AI review 给我打分太低我不服怎么办?

A: 申诉流程：在 PR 里 @ 讲师，并简述哪一条判错。讲师会人工复核。

附录 C：Troubleshooting

现象	可能原因	解决
<code>claude</code> 启动卡住	网络问题 / 登录失效	试 <code>claude logout</code> 再 <code>claude login</code>
<code>claude --version</code> 报错	npm 安装路径不对	检查 <code>which claude</code> ，确认指向 npm 全局路径
<code>rtk gain</code> 报错 "command not found"	rtk 没装 / 装错了同名工具	见 ~/.claude/RTK.md 警告（有同名工具 reachingforthejack/rtk）
<code>gh repo create</code> 失败	没登录 gh	<code>gh auth login</code>
创建 repo 后 push 报 403	gh 权限不够	<code>gh auth refresh -s repo</code>
Claude 写出的代码跑不起来	上下文不够 / CLAUDE.md 太空	先让 Claude 帮你看错误信息，再补 CLAUDE.md
<code>codex</code> 装不上	npm 镜像问题	换 registry 或用 yarn

更多 troubleshooting：周五 lab 直接问讲师/助教。

周四下课前 Check-in

请每人在群里回复：

1. 你打算做什么项目（一句话）
2. 你最担心 W1 的哪一步
3. 你想问的一个问题

讲师会在周四晚之前回复所有担心点。

周五下课前 Check-in

请每人在群里回复：

1. 你的 repo URL
 2. 你 hello-world 是否跑通（是 / 否 / 卡在哪）
 3. 工具箱 6 个有没有装齐
 4. 一句话 retrospective："这一周我最爽的瞬间是 _ / 最痛苦的瞬间是 _"
-

下周预告：W2 我们正式进入 spec-driven 开发——写 PRD、用 spec-kit 拆 issue、并造你的第一个 custom skill。

别忘了：每周三 23:59 前提交 `report.md`。这是从 W1 开始就要做的事，不是 W2 才开始。